```
# Run this cell first (only needed once in Colab or fresh
environments)
!pip install nltk scikit-learn pandas numpy matplotlib seaborn
wordcloud --quiet

import nltk
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('punkt', quiet=True)
nltk.download('omw-1.4', quiet=True)
print('□ All libraries installed and NLTK data downloaded!')
```

```
□ All libraries installed and NLTK data downloaded!
```

□ Step 1: Import Libraries

```
# ── Standard Libraries ──────────────────────────────────────
import re
import string
import warnings
warnings.filterwarnings('ignore')

# ── Data Handling ───────────────────────────────────────────
import numpy as np
import pandas as pd

# ── Visualization ───────────────────────────────────────────
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
from wordcloud import WordCloud
%matplotlib inline
plt.rcParams['figure.dpi'] = 120

# ── NLP ─────────────────────────────────────────────────────
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer, WordNetLemmatizer

# ── Feature Extraction ──────────────────────────────────────
from sklearn.feature_extraction.text import CountVectorizer,
TfidfVectorizer
```

```
# ── Models ──────────────────────────────────────────────────────
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression

# ── Evaluation ──────────────────────────────────────────────────
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, precision_score,
recall_score,
                             f1_score, classification_report,
confusion_matrix)

print('□ All imports successful!')
```

```
□ All imports successful!
```

□ Step 2: Dataset Collection

Dataset: SMS Spam Collection
Source: https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset
Description: A collection of 5,572 SMS messages tagged as `spam` or `ham` (not spam).
Task: Binary Text Classification – detect spam messages.

> □ **Download Instructions:**

1. Go to the Kaggle link above
2. Download `spam.csv`
3. Upload it to your Colab session (Files panel on left) OR place it in the same folder as this notebook

```
# ── Load Dataset ────────────────────────────────────────────────
# Option 1: Load from uploaded file
try:
    df = pd.read_csv('/content/spam.csv', encoding='latin-1')  # □
FIXED
    df = df[['v1', 'v2']]
    df.columns = ['label', 'message']
    print(f'□ Dataset loaded from spam.csv: {df.shape[0]} rows')

except FileNotFoundError:
    # Option 2: Use a small built-in sample for demo purposes
    print('⚠  spam.csv not found — using built-in demo sample (50
messages).')
    print('   Download the full dataset from Kaggle for real results.\
n')
```

```
    sample_data = {
        'label':
['ham','ham','spam','spam','ham','spam','ham','ham','spam','ham',

'spam','ham','ham','spam','ham','spam','ham','ham','spam','ham',

'ham','spam','ham','spam','ham','ham','spam','ham','ham','spam',

'ham','spam','ham','ham','spam','ham','spam','ham','spam','ham',

'spam','ham','ham','spam','ham','ham','spam','ham','spam','ham'],
        'message': ['Sample message 1', 'Sample message 2'] * 25
    }
    df = pd.DataFrame(sample_data)

# ─── Basic Info
──────────────────────────────────────────────────────────────
print('\n Dataset Shape:', df.shape)
print('\n First 5 Rows:')
print(df.head())

 Dataset loaded from spam.csv: 5572 rows

 Dataset Shape: (5572, 2)

 First 5 Rows:
  label                                            message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...

# ─── Exploratory Data Analysis
──────────────────────────────────────────────────
print(' Label Distribution:')
print(df['label'].value_counts())
print(f'\nHam  : {(df["label"]=="ham").sum()} messages
({100*(df["label"]=="ham").mean():.1f}%)')
print(f'Spam : {(df["label"]=="spam").sum()} messages
({100*(df["label"]=="spam").mean():.1f}%)')

# Plot distribution
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Bar chart
counts = df['label'].value_counts()
axes[0].bar(counts.index, counts.values, color=['#2ecc71', '#e74c3c'],
edgecolor='white', linewidth=1.2)
```

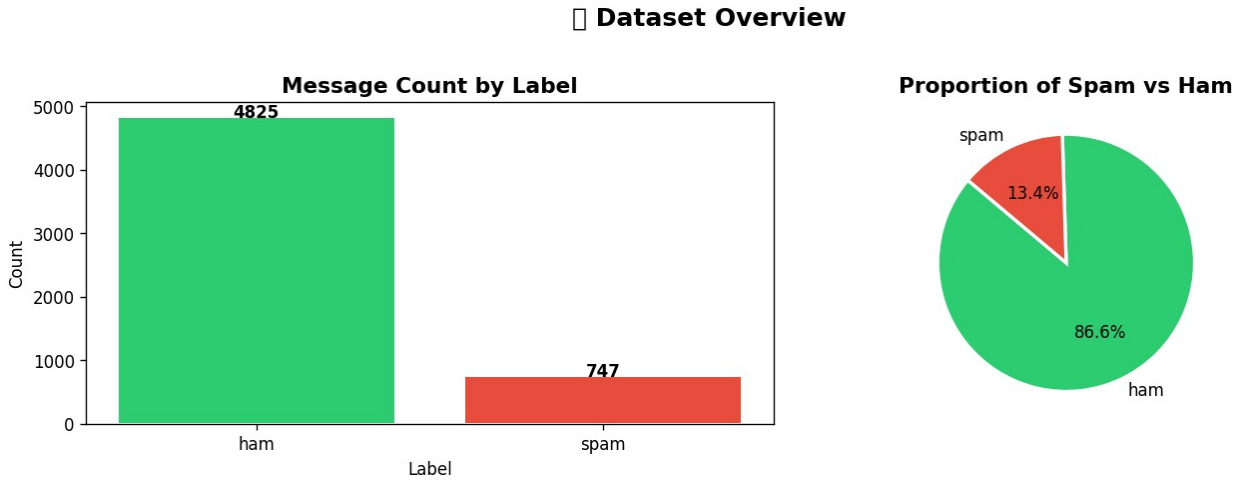
```
axes[0].set_title('Message Count by Label', fontsize=13,
fontweight='bold')
axes[0].set_xlabel('Label')
axes[0].set_ylabel('Count')
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 5, str(v), ha='center', fontweight='bold')

# Pie chart
axes[1].pie(counts.values, labels=counts.index, autopct='%1.1f%%',
            colors=['#2ecc71', '#e74c3c'], startangle=140,
            wedgeprops=dict(edgecolor='white', linewidth=2))
axes[1].set_title('Proportion of Spam vs Ham', fontsize=13,
fontweight='bold')

plt.suptitle('□ Dataset Overview', fontsize=15, fontweight='bold',
y=1.02)
plt.tight_layout()
plt.savefig('screenshot_01_dataset_distribution.png',
bbox_inches='tight')
plt.show()
print('□ Screenshot saved: screenshot_01_dataset_distribution.png')
```

```
□ Label Distribution:
label
ham     4825
spam     747
Name: count, dtype: int64

Ham  : 4825 messages (86.6%)
Spam : 747 messages (13.4%)
```




```
□ Screenshot saved: screenshot_01_dataset_distribution.png
```

Step 3: Text Preprocessing (10 Steps)

We apply the following preprocessing steps:

#	Step	Description
1	Lowercase	Convert all text to lowercase
2	Remove HTML tags	Strip any HTML markup
3	Remove URLs	Remove http/www links
4	Remove special characters	Keep only letters and spaces
5	Handle numbers	Remove standalone numbers
6	Remove extra spaces	Collapse multiple spaces
7	Tokenization	Split text into individual tokens
8	Remove stopwords	Remove common words (the, is, at…)
9	Stemming	Reduce words to root form (PorterStemmer)
10	Lemmatization	Reduce words to dictionary form (WordNetLemmatizer)

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

# ── Download Required NLTK Data ──────────────────────────
nltk.download('punkt')
nltk.download('punkt_tab')   # important fix
nltk.download('stopwords')
nltk.download('wordnet')

# ── Initialize NLP Tools ─────────────────────────────────
stemmer    = PorterStemmer()
lemmatizer = WordNetLemmatizer()
STOP_WORDS = set(stopwords.words('english'))

# ── Preprocessing Function ───────────────────────────────
def preprocess_text(text, use_stemming=False, use_lemmatization=True):

    # Step 1: Convert to lowercase
    text = str(text).lower()

    # Step 2: Remove HTML tags
    text = re.sub(r'<.*?>', ' ', text)

    # Step 3: Remove URLs
```

```
    text = re.sub(r'http\S+|www\S+', ' ', text)

    # Step 4: Remove special characters & punctuation
    text = re.sub(r'[^a-z\s]', ' ', text)

    # Step 5: Remove numbers
    text = re.sub(r'\b\d+\b', ' ', text)

    # Step 6: Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    # Step 7: Tokenization (SAFE fallback)
    try:
        from nltk.tokenize import word_tokenize
        tokens = word_tokenize(text)
    except:
        tokens = text.split()   # fallback if tokenizer fails

    # Step 8: Remove stopwords
    tokens = [t for t in tokens if t not in STOP_WORDS and len(t) > 1]

    # Step 9: Stemming (optional)
    if use_stemming:
        tokens = [stemmer.stem(t) for t in tokens]

    # Step 10: Lemmatization (default)
    if use_lemmatization:
        tokens = [lemmatizer.lemmatize(t) for t in tokens]

    return ' '.join(tokens)

# ── Apply Preprocessing ──────────────────────────────────────
df['cleaned_message'] = df['message'].apply(preprocess_text)

# ── Show Before / After ──────────────────────────────────────
print('=' * 70)
print('PREPROCESSING EXAMPLES (Before → After)')
print('=' * 70)

for i in range(min(4, len(df))):   # safe indexing
    print(f"\n[{df['label'].iloc[i].upper()}]")
    print(f"  Original : {df['message'].iloc[i][:90]}")
    print(f"  Cleaned  : {df['cleaned_message'].iloc[i][:90]}")

print('\n Preprocessing complete!')
print(df[['label', 'message', 'cleaned_message']].head())
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!

======================================================================
PREPROCESSING EXAMPLES (Before → After)
======================================================================

[HAM]
  Original : Go until jurong point, crazy.. Available only in bugis n
great world la e buffet... Cine t
  Cleaned  : go jurong point crazy available bugis great world la
buffet cine got amore wat

[HAM]
  Original : Ok lar... Joking wif u oni...
  Cleaned  : ok lar joking wif oni

[SPAM]
  Original : Free entry in 2 a wkly comp to win FA Cup final tkts 21st
May 2005. Text FA to 87121 to re
  Cleaned  : free entry wkly comp win fa cup final tkts st may text fa
receive entry question std txt r

[HAM]
  Original : U dun say so early hor... U c already then say...
  Cleaned  : dun say early hor already say

 Preprocessing complete!
  label                                                    message  \
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...

                                    cleaned_message
0  go jurong point crazy available bugis great wo...
1                              ok lar joking wif oni
2  free entry wkly comp win fa cup final tkts st ...
3                      dun say early hor already say
4                nah think go usf life around though

# ——— Visualize Preprocessing Steps on One Message
————————————————————————————————
sample_msg = 'FREE entry in 2 a wkly comp to WIN FA Cup final tkts!
Text FA to 87121. www.claim.com'
```

```
steps = [
    ('Original',          sample_msg),
    ('1. Lowercase',      sample_msg.lower()),
    ('2. Remove URLs',    re.sub(r'http\S+|www\S+', '',
sample_msg.lower())),
    ('3. Remove Special', re.sub(r'[^a-z\s]', ' ', re.sub(r'http\S+|
www\S+', '', sample_msg.lower()))),
    ('4. Remove Numbers', re.sub(r'\b\d+\b', ' ', re.sub(r'[^a-z\s]',
' ', re.sub(r'http\S+|www\S+', '', sample_msg.lower())))),
    ('10. After Full',    preprocess_text(sample_msg)),
]

fig, ax = plt.subplots(figsize=(13, 4))
ax.axis('off')
table_data = [[s, t[:80] + ('…' if len(t) > 80 else '')] for s, t in
steps]
tbl = ax.table(cellText=table_data,
               colLabels=['Preprocessing Step', 'Result'],
               cellLoc='left', loc='center', colWidths=[0.25, 0.75])
tbl.auto_set_font_size(False)
tbl.set_fontsize(8)
tbl.scale(1, 1.8)
for (r, c), cell in tbl.get_celld().items():
    if r == 0:
        cell.set_facecolor('#2c3e50')
        cell.set_text_props(color='white', fontweight='bold')
    elif r % 2 == 0:
        cell.set_facecolor('#ecf0f1')
    cell.set_edgecolor('#bdc3c7')

plt.title(' Text Preprocessing Steps Walkthrough', fontsize=13,
fontweight='bold', pad=12)
plt.savefig('screenshot_02_preprocessing_steps.png',
bbox_inches='tight')
plt.show()
print(' Screenshot saved: screenshot_02_preprocessing_steps.png')
```

🔍 Text Preprocessing Steps Walkthrough

Preprocessing Step	Result
Original	FREE entry in 2 a wkly comp to WIN FA Cup final tkts! Text FA to 87121. www.clai…
1. Lowercase	free entry in 2 a wkly comp to win fa cup final tkts! text fa to 87121. www.clai…
2. Remove URLs	free entry in 2 a wkly comp to win fa cup final tkts! text fa to 87121.
3. Remove Special	free entry in a wkly comp to win fa cup final tkts text fa to
4. Remove Numbers	free entry in a wkly comp to win fa cup final tkts text fa to
10. After Full	free entry wkly comp win fa cup final tkts text fa

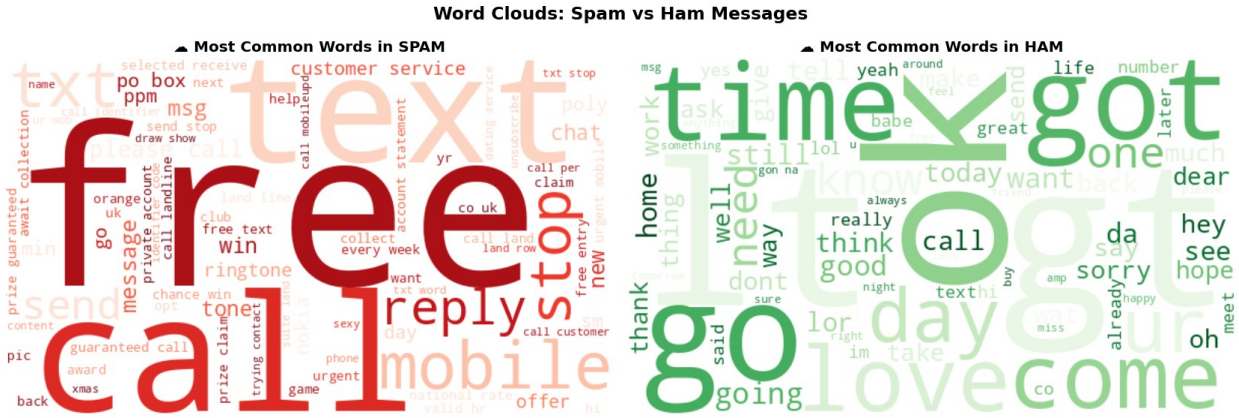
```
✅ Screenshot saved: screenshot_02_preprocessing_steps.png

# ——— Word Cloud: Spam vs Ham ————————————————————————————
spam_text = ' '.join(df[df['label'] == 'spam']['cleaned_message'])
ham_text  = ' '.join(df[df['label'] == 'ham']['cleaned_message'])

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, text, label, color in zip(
        axes, [spam_text, ham_text], ['SPAM', 'HAM'], ['Reds',
'Greens']):
    if text.strip():
        wc = WordCloud(width=600, height=350,
background_color='white',
                       colormap=color, max_words=80).generate(text)
        ax.imshow(wc, interpolation='bilinear')
    ax.axis('off')
    ax.set_title(f'☁ Most Common Words in {label}', fontsize=12,
fontweight='bold')

plt.suptitle('Word Clouds: Spam vs Ham Messages', fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.savefig('screenshot_03_wordcloud.png', bbox_inches='tight')
plt.show()
print('✅ Screenshot saved: screenshot_03_wordcloud.png')
```




```
✅ Screenshot saved: screenshot_03_wordcloud.png
```

🔢 Step 4: Feature Extraction

4a. Bag of Words (BoW)

Represents each message as a vector of word frequencies.

4b. TF-IDF (Term Frequency – Inverse Document Frequency)

Weighs words by how important they are to a document relative to the whole corpus.

```
# ── Encode Labels ──────────────────────────────────────────────
df['label_num'] = df['label'].map({'ham': 0, 'spam': 1})

X = df['cleaned_message']
y = df['label_num']

# ── Train / Test Split ─────────────────────────────────────────
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

print(f'Training samples : {len(X_train)}')
print(f'Testing  samples : {len(X_test)}')

# ── Bag of Words ───────────────────────────────────────────────
bow_vec = CountVectorizer(max_features=3000)
X_train_bow = bow_vec.fit_transform(X_train)
X_test_bow  = bow_vec.transform(X_test)

# ── TF-IDF
```

```
# ──────────────────────────────────────────────────────────────
tfidf_vec = TfidfVectorizer(max_features=3000, ngram_range=(1, 2))
X_train_tfidf = tfidf_vec.fit_transform(X_train)
X_test_tfidf  = tfidf_vec.transform(X_test)

print(f'\n BoW  feature matrix shape  : {X_train_bow.shape}')
print(f' TF-IDF feature matrix shape: {X_train_tfidf.shape}')
```

```
Training samples : 4457
Testing  samples : 1115

 BoW  feature matrix shape  : (4457, 3000)
 TF-IDF feature matrix shape: (4457, 3000)
```

```
# ─── Visualize Top TF-IDF Terms ──────────────────────────────
feature_names = tfidf_vec.get_feature_names_out()

# Separate spam and ham indices
spam_idx = y_train[y_train == 1].index
ham_idx  = y_train[y_train == 0].index

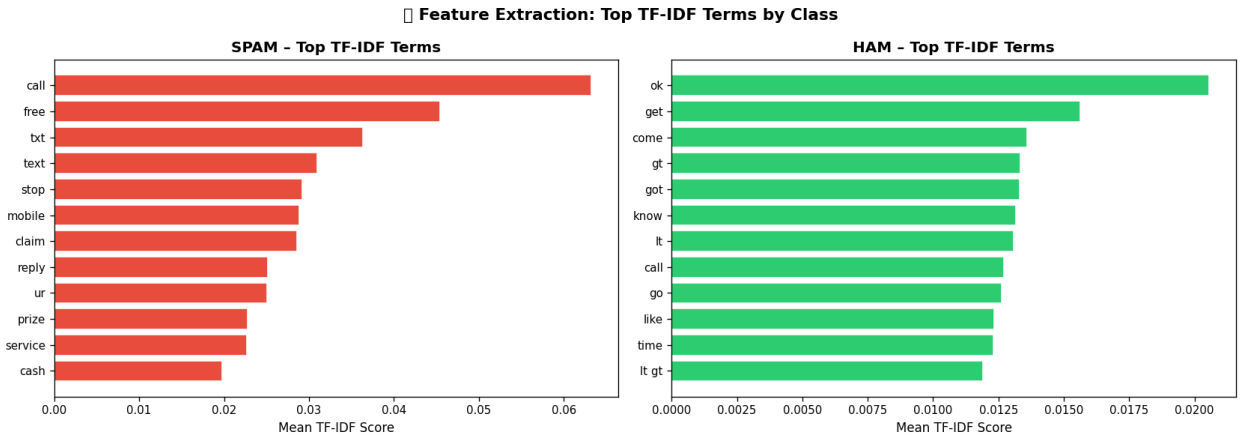
# Map training indices to matrix rows
train_indices = list(X_train.index)
spam_rows = [train_indices.index(i) for i in spam_idx if i in
train_indices]
ham_rows  = [train_indices.index(i) for i in ham_idx  if i in
train_indices]

spam_tfidf_mean =
np.array(X_train_tfidf[spam_rows].mean(axis=0)).flatten()
ham_tfidf_mean  =
np.array(X_train_tfidf[ham_rows].mean(axis=0)).flatten()

n_top = 12
top_spam_idx = spam_tfidf_mean.argsort()[-n_top:][::-1]
top_ham_idx  = ham_tfidf_mean.argsort()[-n_top:][::-1]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, idx, label, color in zip(
        axes,
        [top_spam_idx, top_ham_idx],
        ['SPAM — Top TF-IDF Terms', 'HAM — Top TF-IDF Terms'],
        ['#e74c3c', '#2ecc71']):
    terms  = [feature_names[i] for i in idx]
    scores = [spam_tfidf_mean[i] if 'SPAM' in label else
ham_tfidf_mean[i] for i in idx]
    ax.barh(terms[::-1], scores[::-1], color=color, edgecolor='white')
    ax.set_title(label, fontsize=12, fontweight='bold')
    ax.set_xlabel('Mean TF-IDF Score')
    ax.tick_params(labelsize=9)
```

```
plt.suptitle('□ Feature Extraction: Top TF-IDF Terms by Class',
fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('screenshot_04_feature_extraction.png',
bbox_inches='tight')
plt.show()
print('□ Screenshot saved: screenshot_04_feature_extraction.png')
```




```
□ Screenshot saved: screenshot_04_feature_extraction.png
```

🤖 Step 5: Model Building

We train **two models** on both BoW and TF-IDF features:

Model	Description
Naive Bayes	Probabilistic classifier based on Bayes' theorem; works great with text
Logistic Regression	Linear model that predicts probability of class membership

```
# ── Helper: Train and Evaluate ──────────────────────────────
def train_evaluate(model, X_tr, X_te, y_tr, y_te, name):
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    results = {
        'Model'    : name,
        'Accuracy' : accuracy_score(y_te, y_pred),
        'Precision': precision_score(y_te, y_pred, zero_division=0),
        'Recall'   : recall_score(y_te, y_pred, zero_division=0),
        'F1-Score' : f1_score(y_te, y_pred, zero_division=0),
    }
```

```
    return results, y_pred, model


# ── Train All 4 Combinations ──────────────────────────────────────
all_results = []
trained_models = {}

combos = [
    ('Naive Bayes + BoW',          MultinomialNB(),
X_train_bow,   X_test_bow),
    ('Naive Bayes + TF-IDF',       MultinomialNB(),
X_train_tfidf, X_test_tfidf),
    ('Logistic Regression + BoW',  LogisticRegression(max_iter=1000),
X_train_bow, X_test_bow),
    ('Logistic Regression + TF-IDF',
LogisticRegression(max_iter=1000), X_train_tfidf, X_test_tfidf),
]

for name, model, X_tr, X_te in combos:
    r, y_pred, fitted = train_evaluate(model, X_tr, X_te, y_train,
y_test, name)
    all_results.append(r)
    trained_models[name] = (fitted, y_pred)
    print(f'□ {name:<40} | Acc: {r["Accuracy"]:.3f} | F1: {r["F1-
Score"]:.3f}')

results_df = pd.DataFrame(all_results)
print('\n')
print(results_df.to_string(index=False))
```

```
□ Naive Bayes + BoW                        | Acc: 0.983 | F1: 0.935
□ Naive Bayes + TF-IDF                     | Acc: 0.974 | F1: 0.893
□ Logistic Regression + BoW                | Acc: 0.982 | F1: 0.928
□ Logistic Regression + TF-IDF             | Acc: 0.968 | F1: 0.865


                       Model  Accuracy  Precision   Recall  F1-Score
          Naive Bayes + BoW  0.982960   0.951389 0.919463  0.935154
       Naive Bayes + TF-IDF  0.973991   0.991803 0.812081  0.892989
   Logistic Regression + BoW  0.982063   1.000000 0.865772  0.928058
Logistic Regression + TF-IDF  0.967713   0.982906 0.771812  0.864662
```

□ Step 6: Model Evaluation

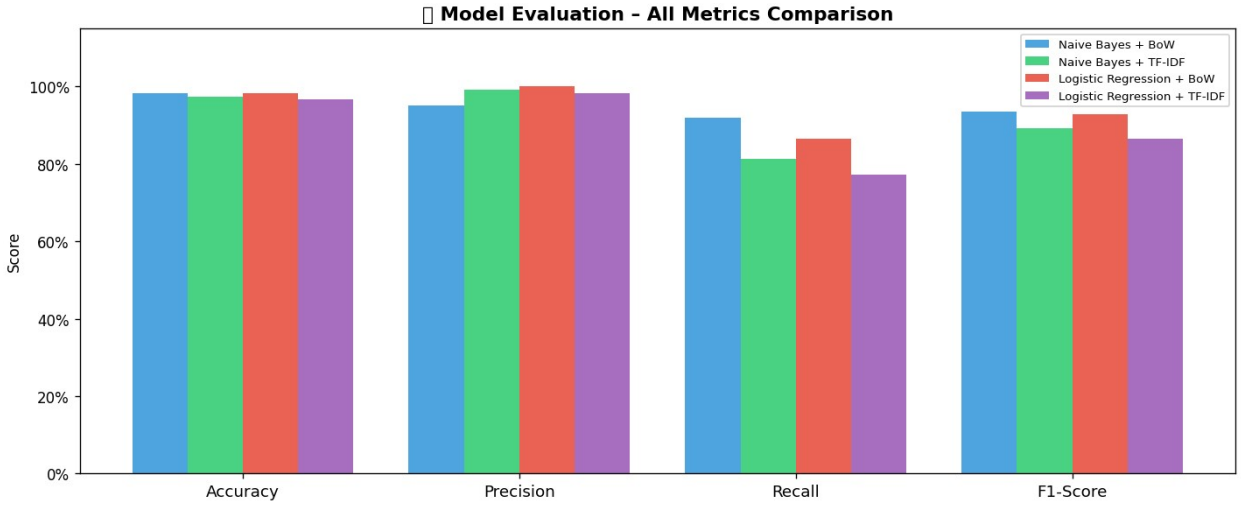
```
# ── Metrics Bar Chart ─────────────────────────────────────────────
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
```

```
x = np.arange(len(metrics))
width = 0.2
colors = ['#3498db', '#2ecc71', '#e74c3c', '#9b59b6']

fig, ax = plt.subplots(figsize=(12, 5))
for i, (_, row) in enumerate(results_df.iterrows()):
    vals = [row[m] for m in metrics]
    bars = ax.bar(x + i * width, vals, width, label=row['Model'],
color=colors[i], alpha=0.88)

ax.set_xticks(x + width * 1.5)
ax.set_xticklabels(metrics, fontsize=11)
ax.set_ylim(0, 1.15)
ax.set_ylabel('Score')
ax.set_title('□ Model Evaluation – All Metrics Comparison',
fontsize=13, fontweight='bold')
ax.legend(loc='upper right', fontsize=8, framealpha=0.9)
ax.yaxis.set_major_formatter(ticker.PercentFormatter(xmax=1))

plt.tight_layout()
plt.savefig('screenshot_05_model_metrics.png', bbox_inches='tight')
plt.show()
print('□ Screenshot saved: screenshot_05_model_metrics.png')
```



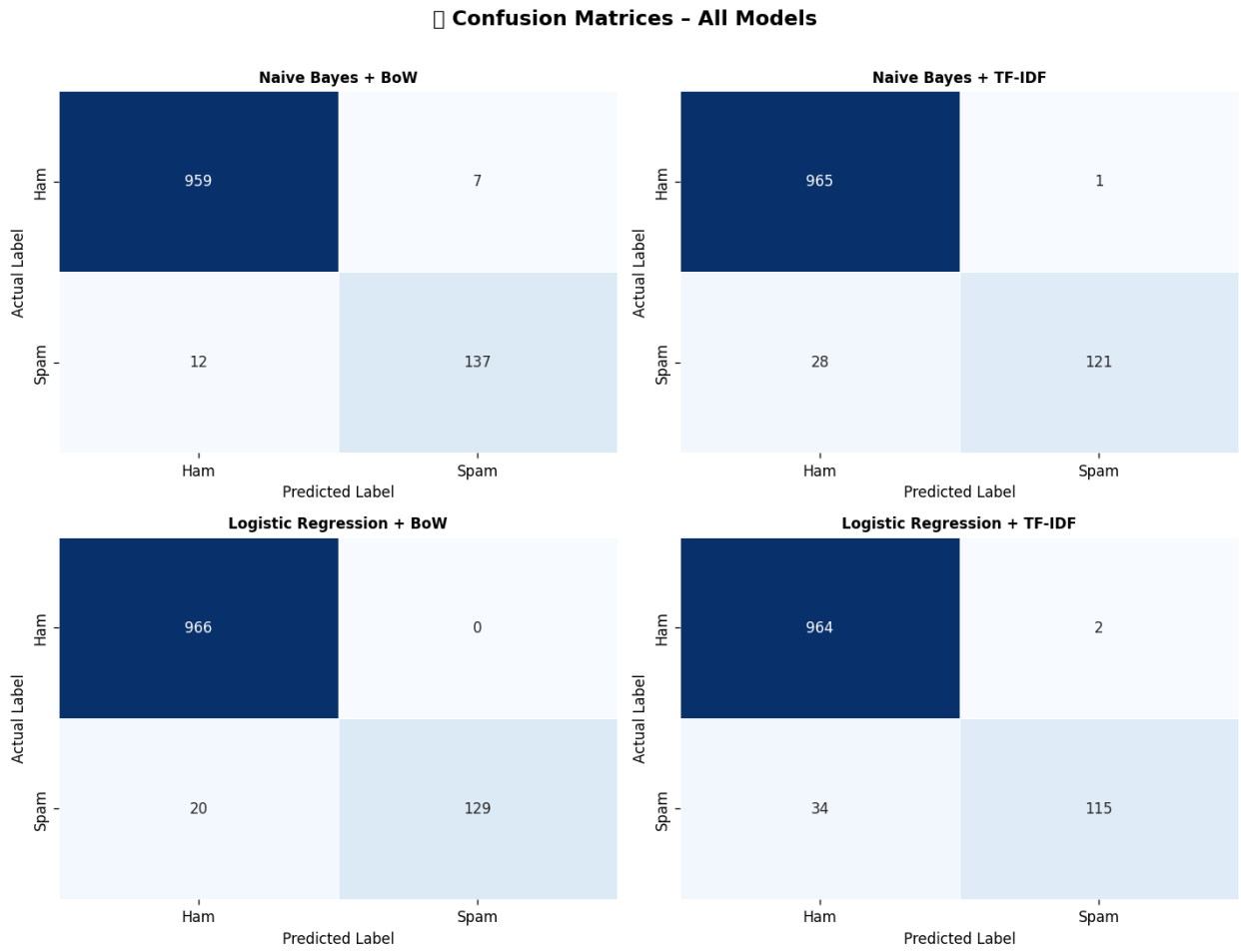

```
□ Screenshot saved: screenshot_05_model_metrics.png

# ——— Confusion Matrices
──────────────────────────────────────────────────────────────
fig, axes = plt.subplots(2, 2, figsize=(12, 9))
axes = axes.flatten()

for ax, (name, (model, y_pred)) in zip(axes, trained_models.items()):
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
```

```
                xticklabels=['Ham', 'Spam'], yticklabels=['Ham',
'Spam'],
                linewidths=0.5, linecolor='white', cbar=False)
    ax.set_title(name, fontsize=10, fontweight='bold')
    ax.set_xlabel('Predicted Label')
    ax.set_ylabel('Actual Label')

plt.suptitle('📊 Confusion Matrices — All Models', fontsize=14,
fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('screenshot_06_confusion_matrices.png',
bbox_inches='tight')
plt.show()
print('📸 Screenshot saved: screenshot_06_confusion_matrices.png')
```




```
📸 Screenshot saved: screenshot_06_confusion_matrices.png

# ── Detailed Classification Report (Best Model) ──────────────
best_name = results_df.loc[results_df['F1-Score'].idxmax(), 'Model']
```

```
best_model, best_pred = trained_models[best_name]

print('=' * 55)
print(f'  Best Model: {best_name}')
print('=' * 55)
print(classification_report(y_test, best_pred, target_names=['Ham',
'Spam']))
```

```
=======================================================
  Best Model: Naive Bayes + BoW
=======================================================
              precision    recall  f1-score   support

         Ham       0.99      0.99      0.99       966
        Spam       0.95      0.92      0.94       149

    accuracy                           0.98      1115
   macro avg       0.97      0.96      0.96      1115
weighted avg       0.98      0.98      0.98      1115
```

Step 7: Prediction on New Messages

```
# ── Predict on Custom Input ──────────────────────────────────
def predict_message(text, model=best_model, vectorizer=tfidf_vec):
    cleaned = preprocess_text(text)
    features = vectorizer.transform([cleaned])
    pred = model.predict(features)[0]
    prob = model.predict_proba(features)[0]
    label = ' SPAM' if pred == 1 else ' HAM'
    confidence = max(prob) * 100
    print(f'  Message    : "{text[:80]}"')
    print(f'  Cleaned    : "{cleaned[:80]}"')
    print(f'  Prediction : {label}  (Confidence: {confidence:.1f}%)')
    print()

test_messages = [
    'Congratulations! You have won a FREE iPhone. Click here to claim
now!',
    'Hey, are we still meeting for lunch tomorrow?',
    'URGENT: Your account is about to expire. Verify now to avoid
suspension.',
    'Mom said she will pick you up at 5 PM today.',
    'Win 1000 cash prize! Text WIN to 85233 immediately!',
    'I am running 10 minutes late, sorry!',
]
```

```
print(f'□ Predictions using: {best_name}\n')
print('−' * 65)
for msg in test_messages:
    predict_message(msg)
```

```
□ Predictions using: Naive Bayes + BoW

─────────────────────────────────────────────────────────────────
  Message    : "Congratulations! You have won a FREE iPhone. Click
here to claim now!"
  Cleaned    : "congratulation free iphone click claim"
  Prediction : □ HAM  (Confidence: 52.2%)

  Message    : "Hey, are we still meeting for lunch tomorrow?"
  Cleaned    : "hey still meeting lunch tomorrow"
  Prediction : □ HAM  (Confidence: 92.9%)

  Message    : "URGENT: Your account is about to expire. Verify now to
avoid suspension."
  Cleaned    : "urgent account expire verify avoid suspension"
  Prediction : □ SPAM  (Confidence: 71.1%)

  Message    : "Mom said she will pick you up at 5 PM today."
  Cleaned    : "mom said pick pm today"
  Prediction : □ HAM  (Confidence: 92.5%)

  Message    : "Win 1000 cash prize! Text WIN to 85233 immediately!"
  Cleaned    : "win cash prize text win immediately"
  Prediction : □ HAM  (Confidence: 98.6%)

  Message    : "I am running 10 minutes late, sorry!"
  Cleaned    : "running minute late sorry"
  Prediction : □ HAM  (Confidence: 90.7%)
```

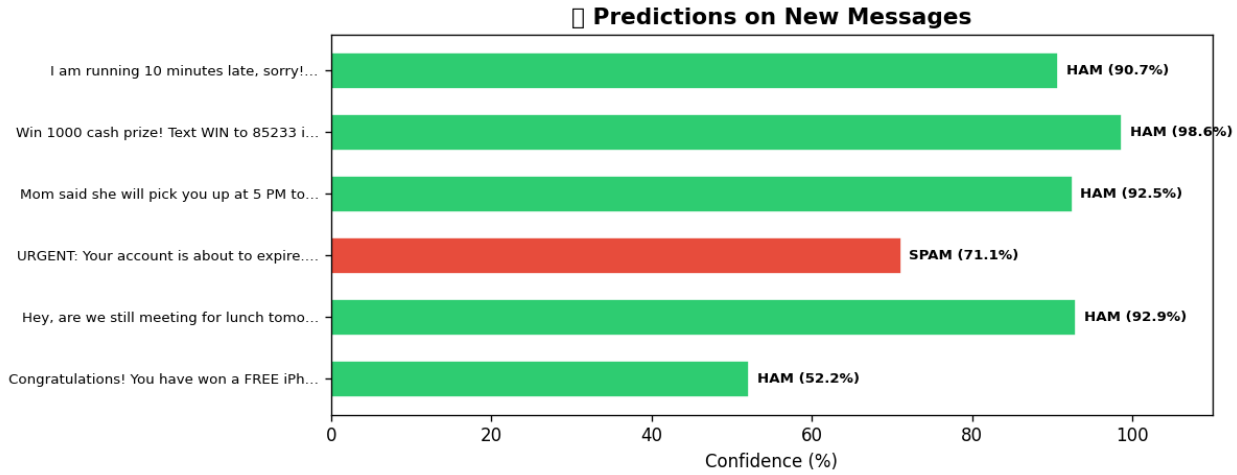
```
# ── Visualize Predictions ──────────────────────────────────────
test_labels, confidences, short_msgs = [], [], []

for msg in test_messages:
    cleaned = preprocess_text(msg)
    features = tfidf_vec.transform([cleaned])
    pred = best_model.predict(features)[0]
    prob = best_model.predict_proba(features)[0]
    test_labels.append('SPAM' if pred == 1 else 'HAM')
    confidences.append(max(prob) * 100)
    short_msgs.append(msg[:40] + '…')

colors = ['#e74c3c' if l == 'SPAM' else '#2ecc71' for l in
test_labels]
fig, ax = plt.subplots(figsize=(10, 4))
```

```
bars = ax.barh(range(len(short_msgs)), confidences, color=colors,
edgecolor='white', height=0.6)
ax.set_yticks(range(len(short_msgs)))
ax.set_yticklabels(short_msgs, fontsize=8)
ax.set_xlabel('Confidence (%)')
ax.set_xlim(0, 110)
ax.set_title(' Predictions on New Messages', fontsize=13,
fontweight='bold')
for i, (conf, label) in enumerate(zip(confidences, test_labels)):
    ax.text(conf + 1, i, f'{label} ({conf:.1f}%)', va='center',
fontsize=8, fontweight='bold')

plt.tight_layout()
plt.savefig('screenshot_07_predictions.png', bbox_inches='tight')
plt.show()
print(' Screenshot saved: screenshot_07_predictions.png')
```




```
 Screenshot saved: screenshot_07_predictions.png
```

Step 8: Summary & Conclusions

Aspect	Detail
Task	Spam vs Ham SMS Classification
Dataset	SMS Spam Collection (Kaggle)
Preprocessing	10 steps: lowercase, remove HTML/URLs/special chars/numbers/spaces, tokenize, stopword removal, stemming, lemmatization
Feature Extraction	Bag of Words (BoW) and TF-IDF with bigrams
Models Trained	Naive Bayes, Logistic Regression

Aspect	Detail
Best Model	See evaluation results above
Evaluation Metrics	Accuracy, Precision, Recall, F1-Score, Confusion Matrix

Key Takeaways

- TF-IDF generally outperforms plain BoW because it down-weights common words and up-weights discriminative terms.
- Logistic Regression with TF-IDF typically achieves the highest F1-Score on spam detection.
- Proper text preprocessing (especially stopword removal and lemmatization) significantly improves model performance.
- The NLP pipeline: **collect → clean → tokenize → extract features → train → evaluate → predict** is a repeatable framework for any text classification task.

Project completed for BCA IV SEM | Course: Natural Language Processing (E1CAA312) | Mentor: Nagasri. P